DEEP LEARNING REVISION GUIDE

A beginner-to-advanced map of neural networks, vision, sequence models, and Transformers

---

How to use this guide

Read in order for a first pass. For revision, use the linked contents and the interview callouts. Shapes are written as (batch, ...); batch size is usually omitted when discussing one example. Code snippets are compact TensorFlow/Keras patterns, not a complete training package.

Scope note: This is a revision/reference guide, not exhaustive Deep Learning coverage. It focuses on the completed topics listed for this study path; implementation details can vary by TensorFlow/Keras version and task.

Core mental model

Represent data as tensors → **compute predictions** → **measure error** → **use gradients to adjust parameters** → **repeat.** Architectures differ in how they represent structure and share computation.

Contents

Select a topic to jump directly to it. PDF bookmarks mirror these section headings.

1. Neural networks and ANN fundamentals
2. Activations, losses, and optimizers
3. Gradient descent and the learning rate
4. Forward propagation and tensor shapes
5. Backpropagation and the chain rule
6. TensorFlow GradientTape and a custom loop
7. Keras Sequential vs Functional API
8. CNN fundamentals: filters and feature maps
9. Padding, stride, and output size
10. MNIST CNN project: end-to-end workflow
11. Batch Normalization
12. RNN: sequences and recurrent state
13. LSTM and GRU
14. Words: one-hot, Word2Vec, and Embedding
15. Tokenization, padding, and OOV handling
16. Attention: Q, K, and V
17. Multi-head attention: numerical intuition
18. Transformer architecture
19. Positional encoding: sine and cosine
20. BERT vs GPT: objectives and internals
21. MLM, next-token prediction, logits, and sampling
22. BERT vs GPT internals: a compact comparison
23. KV cache: faster autoregressive generation
24. Final checklist and mental model

1. Neural networks and ANN fundamentals

An artificial neural network (ANN) composes simple transformations. A dense layer maps an input vector to a new representation; stacking layers lets the model build increasingly useful features.

```
z = W x + b
a = phi(z)
# W: learned weights; b: learned bias; phi: activation
```

For a batch, the common dense-layer shapes are input (B, d_in), weights (d_in, d_out), bias (d_out,), output (B, d_out). Training changes W and b to reduce a chosen loss.

Interview check: A neural network is not “learning rules” directly. It adjusts parameters so its outputs better match targets under a specified objective and data distribution.

2. Activations, losses, and optimizers

Choice	Typical use	Key point
ReLU: max(0,x)	Hidden layers	Simple, often effective; inactive units can stop updating for some inputs.
Sigmoid	Binary probability	Output in (0,1); use a binary cross-entropy objective.
Softmax	Mutually exclusive classes	Turns class logits into a distribution summing to 1.
MSE	Regression	Penalizes squared residuals; outliers have a strong effect.
Cross-entropy	Classification	Rewards probability assigned to the correct class.
SGD / Adam	Parameter updates	SGD follows gradients; Adam adapts per-parameter step sizes.

The activation introduces nonlinearity. Without nonlinear activations, a stack of affine layers collapses to one affine transformation. A logit is an unnormalized score; softmax converts a vector of logits into class probabilities.

Common pairing: For integer class IDs, sparse categorical cross-entropy is convenient. With logits, configure from_logits=True; with softmax probabilities, use from_logits=False. Avoid applying softmax twice.

3. Gradient descent and the learning rate

Given parameters theta and loss L(theta), gradient descent updates parameters opposite the gradient:

```
theta_(t+1) = theta_t - eta * grad_theta L(theta_t)
```

The learning rate eta sets update scale. Too large can overshoot or diverge; too small can make learning slow. Batch gradient uses all examples, stochastic gradient uses one, and mini-batch gradient uses a subset (the usual practical choice).

An epoch is one pass through the training set. A step is one optimizer update, often after one mini-batch. Validation data estimates generalization during development; keep the test set for final evaluation.

Interview check: A gradient gives a local direction of steepest increase in a scalar objective. Gradient descent uses its negative, but optimization is not guaranteed to find a global minimum in a non-convex network.

4. Forward propagation and tensor shapes

Forward propagation evaluates each operation from inputs to outputs. Track shape at every layer; most shape bugs are mismatches between the expected feature axis and the actual one.

```
x:       (B, 784)
Dense(128): (B, 128)
Dense(10):  (B, 10)  # logits for 10 classes
```

For an image batch in channels-last format: (B, H, W, C). For a token batch: (B, T), where T is sequence length; after embedding: (B, T, D). B, H, W, C, T, and D are symbolic dimensions.

Shape habit: Write down batch, spatial/sequence, and feature/channel axes before and after each operation. The batch axis is typically preserved.

5. Backpropagation and the chain rule

Backpropagation efficiently computes derivatives of a loss with respect to every parameter by applying the chain rule backward through the computational graph.

```
If y = f(g(x)), then dy/dx = (df/dg) * (dg/dx).
For a layer: dL/dW uses the upstream loss gradient and layer input.
Then an optimizer applies the parameter update.
```

For a multilayer network, each layer receives an upstream gradient from later computations, multiplies by its local derivative, and passes the result backward. Automatic differentiation systems build or trace these dependencies.

Distinction: Backpropagation computes gradients; gradient descent or Adam uses those gradients to update parameters. They are related steps, not synonyms.

6. TensorFlow GradientTape and a custom loop

GradientTape records operations involving watched tensors, then differentiates a scalar loss. The example below makes the data flow explicit; production code also needs batching, metrics, validation, and checkpointing.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10)  # logits
])
optimizer = tf.keras.optimizers.Adam(1e-3)
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(
    from_logits=True)

for x_batch, y_batch in train_ds:
    with tf.GradientTape() as tape:
        logits = model(x_batch, training=True)
        loss = loss_fn(y_batch, logits)
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Watch for: The loss must be connected to trainable variables. Use training=True for training-time behavior such as dropout or batch normalization. Custom loops must update metrics and handle validation deliberately.

7. Keras Sequential vs Functional API

API	Use when	Example shape
Sequential	A straight stack where each layer has one input and output.	Input → Dense → Dense
Functional	Branches, merges, shared layers, multiple inputs/outputs.	image input ■■ text input ■■■→ combined outputs

```
inputs = tf.keras.Input(shape=(28, 28, 1))
x = tf.keras.layers.Conv2D(32, 3, activation="relu")(inputs)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
outputs = tf.keras.layers.Dense(10)(x)  # logits
model = tf.keras.Model(inputs, outputs)
```

Use Sequential for readability when the topology is a simple chain. The Functional API creates a graph and is more expressive; both use the same Keras layers and training methods.

8. CNN fundamentals: filters and feature maps

A convolutional neural network exploits local spatial structure. A small learned filter slides across an image; at each position it computes a weighted local response. Early filters may respond to edges or textures; later layers can combine these into task-relevant patterns.

```
image (H,W,C)  --filter kernel-->  feature map (H_out,W_out,F)
```

A convolution kernel has shape (k_h, k_w, C_in, F), where F is the number of filters/output channels. Each filter produces one feature map. Weight sharing means the same filter is applied at many spatial locations, reducing parameters compared with a fully connected layer.

Do not overclaim: A filter is learned from the objective and data. It may resemble an edge detector, but there is no guarantee every filter has a simple human interpretation.

9. Padding, stride, and output size

For one spatial dimension, input size n, kernel size k, padding p on each side, and stride s:

```
n_out = floor((n + 2p - k) / s) + 1
```

Setting	Effect
VALID	No implicit padding; spatial size usually shrinks.
SAME	Pads so output is approximately ceil(n / s); exact padding is distributed around the input.
Stride > 1	Moves the kernel farther each step; output resolution falls.

For a 2D convolution, apply the formula independently to height and width. For SAME with stride 1, output height and width match the input. Pooling uses a similar window/stride idea but typically has no learned filter weights.

Example: n=28, k=3, p=0, s=1 gives floor((28-3)/1)+1 = 26. With SAME and stride 1, it remains 28.

10. MNIST CNN project: end-to-end workflow

MNIST contains 28×28 grayscale handwritten digit images and labels 0–9. A useful project is more than a model: inspect the data, prepare shapes and scales, train reproducibly, then evaluate errors.

Stage	Checklist
EDA	Inspect train/test sizes, label counts, pixel range, example images, and class balance.
Preprocess	Convert to float32, scale pixels consistently (often [0,1]), add channel axis: (N,28,28,1).
Model	Convolutions + pooling, flatten or global pooling, dense classifier; output 10 logits.
Train	Use a held-out validation split, suitable loss/optimizer, batch size, epochs, and random seed.
Callbacks	EarlyStopping on validation loss; ModelCheckpoint saves best weights; ReduceLROnPlateau can lower LR.
Evaluate	Evaluate once on test data; report accuracy and per-class confusion patterns.
Predict	Use model(x, training=False), then argmax over classes; inspect selected predictions.

```
# Compact illustrative Keras model (assumes normalized x with channel axis)
model = tf.keras.Sequential([
    tf.keras.layers.Input((28, 28, 1)),
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10)  # logits
])
model.compile(optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"])
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                     restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True)
]
```

Confusion matrix rows are typically true labels and columns predicted labels (confirm plotting-library convention). The diagonal counts correct predictions; off-diagonal cells expose which digits are confused. Accuracy alone can hide class-specific errors.

Prediction: If the model outputs logits, argmax(logits) gives the predicted class; softmax is unnecessary when only the winning class is needed.

11. Batch Normalization

Batch Normalization (BN) normalizes activations using batch statistics during training, then applies learned scale and offset. It maintains moving estimates used for inference.

```
x = tf.keras.layers.Conv2D(32, 3, use_bias=False)(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.Activation("relu")(x)
```

BN can make optimization less sensitive to activation scale and often works well in CNNs. Its behavior differs between training and inference: training uses current batch statistics; inference uses moving averages. Tiny or highly nonrepresentative batches can make estimates noisy.

Interview check: Call the model with the correct training flag. Keras fit handles this automatically; custom loops must pass training=True for training and False for evaluation.

12. RNN: sequences and recurrent state

A recurrent neural network processes a sequence step by step, carrying a hidden state that summarizes prior inputs.

```
h_t = phi(W_x x_t + W_h h_(t-1) + b)
sequence: x_1, x_2, ... x_T  →  h_1, h_2, ... h_T
```

RNNs share parameters across time and can handle variable-length sequences (with masking/padding conventions). Basic RNNs struggle with long-range dependencies because repeated gradient multiplication can cause vanishing or exploding gradients. LSTM and GRU gates help control information flow.

Shape: With batch_first style conventions in Keras, input often has shape (B,T,D). A sequence-returning RNN yields (B,T,H); a final-state output yields (B,H).

13. LSTM and GRU

Cell	Main components	Intuition
LSTM	Input, forget, output gates; cell state.	Separate cell memory path can preserve information over longer spans.
GRU	Update and reset gates; no separate cell state.	A simpler gated design that blends old state and new candidate.

Gates are learned, input-dependent controls, commonly produced with sigmoid activations in [0,1]. They regulate what to retain, discard, or expose. Both models can capture sequential patterns; which performs better depends on data, compute, and setup.

Do not memorize as magic: Gating improves the path for information and gradients, but does not guarantee perfect long-context memory or superior results on every task.

14. Words: one-hot, Word2Vec, and Embedding

Representation	What it stores	Tradeoff
One-hot	One active vocabulary index per token.	Sparse and wide; no inherent similarity between words.
Word2Vec	A dense vector learned from context prediction objectives.	Captures distributional similarity in a fixed embedding table.
Keras Embedding	Trainable lookup from token ID to dense vector.	Learns task-specific vectors end-to-end unless initialized otherwise.

With vocabulary V and embedding width D: integer IDs (B,T) become vectors (B,T,D). Embeddings are not automatically contextual: the same token lookup is the same vector regardless of surrounding words. Contextual models such as Transformers update representations using attention.

Terminology: Word2Vec is a family of predictive embedding methods (commonly skip-gram or CBOW). A Keras Embedding layer is a lookup table; it is not itself the Word2Vec algorithm.

15. Tokenization, padding, and OOV handling

Text models usually tokenize text into words, subwords, or bytes and map tokens to integer IDs. Subword tokenizers can represent rare or unseen words using smaller known units.

```
tokens:   ["deep", "learning"]  → ids: [17, 42]
padded:   [17, 42, 0, 0]         → fixed length T
mask:     [1,  1,  0, 0]         → valid vs padding
```

Padding makes batches rectangular. Truncation limits sequence length. An OOV token or subword fallback handles unseen strings. Padding and OOV IDs must be distinguished consistently; attention masks prevent padded positions from being treated as content where supported.

Practical check: Ensure tokenizer vocabulary, reserved IDs, padding side, truncation policy, and mask semantics match the model and serving pipeline.

16. Attention: Q, K, and V

Attention lets each position build a weighted mixture of information from other positions. Queries ask what to retrieve; keys describe what each position offers for matching; values carry the content to combine.

```
Q = X W_Q   K = X W_K   V = X W_V
Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k)) V
```

Scaled dot products create compatibility scores. Dividing by sqrt(d_k) helps keep score magnitudes manageable as key dimension grows. Softmax turns scores into weights, and multiplying by V forms a weighted sum. Masks can remove disallowed positions before softmax.

Interview check: Q/K/V are learned projections of representations in self-attention. Cross-attention usually forms Q from one sequence and K/V from another.

17. Multi-head attention: numerical intuition

Each head performs its own projected attention. Multiple heads can learn different relations; their outputs are concatenated and projected back to the model width.

```
head_i = Attention(X W_Q_i, X W_K_i, X W_V_i)
MHA(X) = Concat(head_1, ..., head_h) W_O
```

Step	Illustration
1. Compare	A query scores three keys: [2, 1, 0].
2. Scale/softmax	After scaling, suppose weights are about [0.67, 0.24, 0.09].
3. Mix values	Output = 0.67 V1 + 0.24 V2 + 0.09 V3.
4. Multiple heads	Repeat in learned subspaces, concatenate, then apply W_O.

If model width is d_model and there are h heads, a common setup uses d_k=d_v=d_model/h per head. Implementations may fuse projections into one matrix for efficiency.

Intuition, not a fixed behavior: The example weights explain the arithmetic only. Actual learned attention weights depend on inputs, parameters, masks, scaling, and softmax.

18. Transformer architecture

The original Transformer replaces recurrence with attention and position information, enabling parallel processing across sequence positions during training.

```
Encoder layer: self-attention → residual + norm → feed-forward → residual + norm
Decoder layer: masked self-attention → (optional cross-attention) → feed-forward
```

Residual connections add a sublayer input to its output. Layer normalization stabilizes representations. Position-wise feed-forward networks transform each token vector independently after attention. Positional signals give order information because attention alone is permutation-equivariant.

Encoder-decoder Transformers use encoder representations and decoder cross-attention for sequence-to-sequence tasks. BERT uses an encoder stack; GPT uses a decoder-only stack with causal self-attention.

Mental model: Attention moves information among positions; feed-forward layers transform information within each position; residual paths and normalization help deep stacks train.

19. Positional encoding: sine and cosine

Self-attention by itself does not encode token order. The original sinusoidal scheme adds a position-dependent vector to token embeddings:

```
PE(pos, 2i)   = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

Different dimensions use different wavelengths. The fixed formula supports positions beyond those seen in training, though model quality at long lengths is not guaranteed. Learned position embeddings and relative/rotary schemes are alternatives.

```
import tensorflow as tf
import math

def sinusoidal_positions(length, d_model):
    pos = tf.cast(tf.range(length)[:, None], tf.float32)
    i = tf.cast(tf.range(d_model)[None, :], tf.float32)
    angle = pos / tf.pow(10000.0, (2.0 * tf.floor(i / 2.0)) / d_model)
    even = tf.equal(tf.math.floormod(tf.range(d_model), 2), 0)[None, :]
    return tf.where(even, tf.sin(angle), tf.cos(angle))

# Add (T,D) positions to token embeddings shaped (B,T,D).
pos = sinusoidal_positions(tf.shape(x)[1], tf.shape(x)[2])
x = x + pos[None, :, :]
```

Shape check: Position encoding must match (T,D) of token embeddings and broadcast across batch. In production, mind dynamic dimensions and dtype policies.

20. BERT vs GPT: objectives and internals

Property	BERT-style encoder	GPT-style decoder
Attention	Bidirectional self-attention (within input).	Causal self-attention: each position sees only itself and earlier positions.
Typical pretraining	Masked language modeling (MLM).	Next-token prediction / causal language modeling.
Typical use	Text understanding and contextual representations.	Autoregressive text generation and continuation.
Output use	Task head or token representations.	Next-token logits at each position.

The distinction is not simply “BERT understands, GPT generates.” Both learn contextual representations and can support diverse tasks with appropriate fine-tuning or prompting. The attention mask and training objective create their characteristic behavior.

Why decoder-only? Causal masking lets GPT train next-token prediction across many positions in parallel while ensuring each prediction uses only permitted left context. Generation then samples one token at a time.

21. MLM, next-token prediction, logits, and sampling

Masked language modeling hides selected input tokens and trains a model to predict them from surrounding context. GPT-style causal training predicts the next token at each position using the preceding prefix.

```
logits = model(prefix)              # (..., vocab_size)
probs = softmax(logits / temperature)
next_id = sample(probs)              # or argmax for greedy decoding
```

Logits are unnormalized vocabulary scores. Softmax converts them to probabilities. Temperature T rescales logits: lower T sharpens the distribution; higher T flattens it. T must be positive. Top-k or nucleus (top-p) sampling further restricts candidates. Sampling choices change diversity, not the model’s learned weights.

Causal masking sets attention scores for future positions to a very negative value before softmax, making their probability effectively zero. Padding masks separately exclude padded tokens.

Key distinction: At inference, repeatedly append the selected token and predict again. During training, teacher forcing supplies the known shifted target sequence, so many positions can be trained in parallel.

22. BERT vs GPT internals: a compact comparison

Both use Transformer blocks, token embeddings, position information, residual paths, normalization, and feed-forward layers. Their key structural difference is the self-attention visibility pattern.

```
BERT: [token1 ↔ token2 ↔ token3]  (bidirectional context)
GPT:  [token1 → token2 → token3]  (left-to-right allowed context)
```

BERT commonly uses a classification head for sequence labels or a token head for token-level tasks; its MLM head predicts masked vocabulary items. GPT projects hidden states to vocabulary logits and is trained to predict shifted next tokens. Details vary across model generations and implementations.

Avoid absolutes: Not every BERT checkpoint uses every historical pretraining objective, and modern GPT variants can include architectural details beyond the basic decoder-only picture.

23. KV cache: faster autoregressive generation

At each generation step, attention needs keys and values for prior tokens. Without caching, an implementation may recompute those projections for the entire prefix at every step. A KV cache stores past keys and values per layer and appends only the new token's K and V.

```
Prefill prompt: compute K,V for all prompt tokens and store them.
Decode step t: compute q_t, k_t, v_t for newest token.
Attend q_t to cached [K_1...K_t], [V_1...V_t].
Append k_t,v_t; sample next token; continue.
```

The cache reduces repeated computation for prior positions during decoding, especially for long prompts. It consumes memory proportional to layers × batch × cached length × key/value width. Exact layouts and optimizations vary (for example grouped-query attention or paged caches).

Interview check: KV caching does not change the causal model's intended next-token distribution when implemented correctly. It trades memory for reduced recomputation during inference; it is generally not used to train the full sequence in the same way.

24. Final checklist and mental model

Can you explain…	Self-check
Training loop	How forward pass, loss, gradients, and optimizer update connect?
Shapes	What do batch, channel/feature, spatial, and time axes mean here?
Vision	How do filter count, kernel, stride, padding, and output size relate?
Text	How do IDs, embeddings, masks, and positional information fit together?
Attention	What do Q, K, V, softmax, scaling, and masks each do?
BERT/GPT	How do visibility mask and training objective differ?
Generation	What are logits, temperature, autoregressive decoding, and KV cache?
Evaluation	Why use validation, test data, class-wise metrics, and confusion matrix?

One-line mental model

Data becomes tensors; layers transform tensors; a loss measures mismatch; differentiation assigns responsibility; an optimizer updates parameters; evaluation checks whether the learned behavior generalizes. CNNs encode spatial locality, recurrent cells carry state through time, embeddings map symbols to vectors, and Transformers use attention plus position information to exchange context.

Small final reminders

Final reminders: Match logits/probabilities to the loss; track tensor shapes and masks; keep training, validation, and test roles separate; verify API details for your version; use this guide as a map for deeper derivations, experiments, and papers.